Experiment 11: Q-Learning in Grid World

Python Program

```
# Experiment 11: Q-Learning in Grid World

# 1. Import libraries
import numpy as np
import matplotlib.pyplot as plt

# 2. Define Grid World
rows = 4
cols = 4

start_state = 0
goal_state = 15

# Actions: Up, Down, Left, Right
actions = [0, 1, 2, 3]

action_names = ['Up', 'Down', 'Left', 'Right']

# 3. Initialize Q-table
q_table = np.zeros((16, 4))

# 4. Set parameters
alpha = 0.1
gamma = 0.9
epsilon = 1.0
epsilon_min = 0.01
epsilon_decay = 0.995

episodes = 1000
max_steps = 100

rewards_list = []
success_count = 0

# 5. Function to get next state
def get_next_state(state, action):

    row = state // cols
    col = state % cols

    if action == 0:       # Up
        row = max(0, row - 1)

    elif action == 1:     # Down
        row = min(rows - 1, row + 1)

    elif action == 2:     # Left
        col = max(0, col - 1)

    elif action == 3:     # Right
        col = min(cols - 1, col + 1)

    next_state = row * cols + col

    return next_state

# 6. Q-Learning Training
for episode in range(episodes):

    state = start_state
    total_reward = 0
    reached_goal = False

    for step in range(max_steps):

        # Epsilon-greedy action selection
        if np.random.random() < epsilon:
            action = np.random.choice(actions)
        else:
            action = np.argmax(q_table[state])

        # Get next state
        next_state = get_next_state(state, action)

        # Calculate reward
        if next_state == goal_state:
```

```
            reward = 10
            done = True
        else:
            reward = -1
            done = False

        # Update Q-table
        if done:
            target = reward
        else:
            target = reward + gamma * np.max(q_table[next_state])

        q_table[state, action] = q_table[state, action] + alpha * (
            target - q_table[state, action]
        )

        # Update state
        state = next_state
        total_reward += reward

        if done:
            reached_goal = True
            break

    # Store episode reward
    rewards_list.append(total_reward)

    if reached_goal:
        success_count += 1

    # Reduce exploration rate
    epsilon = max(epsilon_min, epsilon * epsilon_decay)

# 7. Calculate performance
success_rate = (success_count / episodes) * 100

print("Q-Learning Training Completed")
print("--------------------------------")
print("Total Episodes:", episodes)
print("Success Rate:", round(success_rate, 2), "%")
print("Final Epsilon:", round(epsilon, 4))

# 8. Display Q-table
print("\nFinal Q-Table:")
print(np.round(q_table, 2))

# 9. Find Optimal Policy
policy = []

for state in range(16):

    if state == goal_state:
        policy.append('G')
    else:
        best_action = np.argmax(q_table[state])
        policy.append(action_names[best_action])

print("\nOptimal Policy:")

for i in range(rows):
    print(policy[i*cols:(i+1)*cols])

# 10. Plot Cumulative Reward
plt.figure(figsize=(8, 5))

plt.plot(rewards_list)

plt.title("Q-Learning Cumulative Reward")
plt.xlabel("Episodes")
plt.ylabel("Total Reward")
plt.grid()

plt.show()

# 11. Display Optimal Policy as Grid
symbols = ['↑', '↓', '←', '→']
policy_grid = []

for state in range(16):

    if state == goal_state:
        policy_grid.append('G')
```

```
    else:
        best_action = np.argmax(q_table[state])
        policy_grid.append(symbols[best_action])

policy_grid = np.array(policy_grid).reshape(4, 4)

plt.figure(figsize=(6, 6))

plt.imshow(np.zeros((4, 4)), cmap='Blues')

for i in range(4):
    for j in range(4):
        plt.text(
            j, i, policy_grid[i, j],
            ha='center',
            va='center',
            fontsize=20
        )

plt.title("Optimal Policy in Grid World")
plt.xticks(range(4))
plt.yticks(range(4))
plt.grid()

plt.show()

# 12. Final Conclusion
print("\nConclusion:")
print("Q-Learning completed successfully.")
print("The agent learned the optimal policy.")
print("The agent learned to reach the goal state.")
print("Exploration and exploitation were balanced.")
```